ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

AI Agents Development & Agentic Automation

Eng. Fatma Elraey

Lab 1 — Build a Native ReAct Agent with Gemini Tools

Field	Detail
Instructor	Fatma Elraey
Course	AI Agents Development (3-Day Course)
Session	Day 1 of 3 — Lab, 3 Hours
Focus	Hands-on construction of a small native Python agent that exposes typed tools, validates arguments, executes tool calls, and feeds observations back to Gemini.

Before You Start

You will need Python, a Gemini API key, and a small local project folder. The lab deliberately keeps the tools simple so you can focus on the agent loop rather than framework magic.

Today's Lab Plan

1. Set up Gemini and Pydantic
2. Build typed Python tools
3. Test tool contracts directly
4. Implement the manual tool-calling loop
5. Add a second tool and multi-step behavior
6. Harden the agent with limits and errors

Install

```
pip install google-genai python-dotenv pydantic --break-system-packages
```

✓ QUICK CHECK — Before You Start

Q1. Where should GEMINI_API_KEY be stored?

Answer: *In an environment variable or .env file that is excluded from version control.*

Q2. What is the goal of this lab?

Answer: *To make the model choose tools while your Python code validates and executes them.*

01 Secure the Gemini Client

Create the smallest reusable Gemini setup and verify that your API key loads without printing the full secret.

- Create a .env file with GEMINI_API_KEY=...
- Add .env to .gitignore
- Create client.py and initialize genai.Client()
- Run one small text request to verify the connection

Why this matters: Every later exercise depends on a clean client setup. Keeping credentials out of source code prevents accidental key leaks.

Try this

```
# client.py
import os
from dotenv import load_dotenv
from google import genai

load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")
assert api_key, "Missing GEMINI_API_KEY"

client = genai.Client(api_key=api_key)

response = client.models.generate_content(
    model="gemini-3.7-flash",
    contents="Reply with exactly: client ready",
)
print(response.text)
```

Expected result

```
# Instructor example — expected output
OUTPUT  client ready
```

02 Create Typed Tools

Build two narrow business tools: an order lookup and a shipping-cost calculator. Keep the real functions deterministic and easy to test without an LLM.

- Define Pydantic input models
- Use positive IDs and sensible numeric constraints
- Return dictionaries instead of free-form prose
- Write direct Python tests before exposing the tools to Gemini

Why this matters: Tool quality determines agent reliability. If a function is unpredictable before the LLM sees it, connecting a model will not make it safer.

Try this

```
from pydantic import BaseModel, Field

class OrderLookup(BaseModel):
    order_id: int = Field(gt=0)
```

```
class ShippingQuote(BaseModel):
    weight_kg: float = Field(gt=0, le=100)
    zone: str

def get_order(order_id: int) -> dict:
    db = {104: {"status": "delayed", "total": 850}}
    return db.get(order_id, {"status": "not_found"})

def calculate_shipping(weight_kg: float, zone: str) -> dict:
    rate = {"local": 20, "regional": 35}.get(zone.lower(), 50)
    return {"cost": round(weight_kg * rate, 2), "currency": "EGP"}

print(get_order(**OrderLookup(order_id=104).model_dump()))
```

Expected result

```
# Instructor example — expected output
OUTPUT  {'status': 'delayed', 'total': 850}
```

PRO TIP *Stretch: Add an enum-like validation rule so zone only accepts local, regional, or international.*

03 Expose Tools to Gemini

Pass the Python functions to Gemini as tools, but disable automatic execution so you can inspect the model-selected function call yourself.

- Create GenerateContentConfig(tools=[...])
- Disable automatic function calling for the learning exercise
- Ask a question that clearly needs one tool
- Print the chosen function name and extracted arguments

Why this matters: This makes the boundary explicit: the model proposes an action; it has not executed anything yet.

Try this

```
from google.genai import types

config = types.GenerateContentConfig(
    tools=[get_order, calculate_shipping],
    automatic_function_calling=types.AutomaticFunctionCallingConfig(disable=True),
)

response = client.models.generate_content(
    model="gemini-3.7-flash",
    contents="What is the status of order 104?",
    config=config,
)

call = response.candidates[0].content.parts[0].function_call
print(call.name)
print(call.args)
```

Expected result

```
# Instructor example — expected output
OUTPUT  get_order
{'order_id': 104}
```

04 Execute the Tool and Return the Observation

Create a safe dispatch table, execute only registered tools, then send the function result back to Gemini for a user-friendly final answer.

- Map approved function names to Python callables
- Reject unknown tool names
- Execute with extracted arguments inside try/except
- Append the model tool call and your function response to the conversation contents
- Ask Gemini for the final answer

Why this matters: This is the core ReAct execution boundary. Production systems add logging, authorization, retries, timeouts, and human approval here.

Try this

```
TOOLS = {
    "get_order": get_order,
    "calculate_shipping": calculate_shipping,
}

call = response.candidates[0].content.parts[0].function_call
if call.name not in TOOLS:
    raise ValueError(f"Unknown tool: {call.name}")

result = TOOLS[call.name](**call.args)

contents = [
    types.Content(role="user", parts=[types.Part(text="What is the status of order 104?")]),
    response.candidates[0].content,
    types.Content(
        role="user",
        parts=[types.Part.from_function_response(
            name=call.name,
            response={"result": result},
            id=call.id,
        )],
    ),
]

final = client.models.generate_content(
    model="gemini-3.7-flash", contents=contents, config=config
)
print(final.text)
```

Expected result

```
# Instructor example — expected output
OUTPUT  A short answer grounded in the tool result, such as: Order 104 is delayed.
```

05 Turn It into a Small Agent Loop

Wrap the previous exercise in a loop so the model can request another tool after seeing an observation. Stop when it returns text instead of a tool call.

- Use a maximum of 5 steps

- At each step, inspect whether a function call exists
- Execute only whitelisted tools
- Append the observation
- Break and print the final text when no tool call is returned

Why this matters: Multi-step execution is what changes one tool call into an agent loop. The maximum-step guard prevents accidental infinite cycles.

Try this

```
MAX_STEPS = 5
for step in range(MAX_STEPS):
    response = client.models.generate_content(
        model="gemini-3.7-flash", contents=contents, config=config
    )
    part = response.candidates[0].content.parts[0]
    call = part.function_call

    if not call:
        print(response.text)
        break

    if call.name not in TOOLS:
        raise ValueError(f"Blocked tool: {call.name}")

    result = TOOLS[call.name](**call.args)
    contents.append(response.candidates[0].content)
    contents.append(types.Content(
        role="user",
        parts=[types.Part.from_function_response(
            name=call.name, response={"result": result}, id=call.id
        )],
    ))
else:
    raise RuntimeError("Agent exceeded MAX_STEPS")
```

Expected result

```
# Instructor example — expected output
OUTPUT  The loop either returns a final answer or stops after the configured safety limit.
```

PRO TIP *Stretch: Ask a question that requires an order lookup followed by a shipping quote, and inspect the two tool calls.*

06 Final Hardening Checklist

Before calling the agent complete, add the controls a production agent loop needs.

- Validate all tool arguments
- Whitelist tool names
- Set MAX_STEPS
- Catch tool exceptions and convert them to safe observations
- Do not expose secrets in tool outputs
- Log tool name, arguments, duration, and success/failure
- Add approval gates later for destructive or external side-effect tools

Why this matters: Agentic systems are powerful because tools have real effects. Reliability comes from the code around the model, not from trusting the model more.

Try this

```
# Minimum mental checklist
# 1) Is this tool allowed?
# 2) Are its arguments valid?
# 3) Is the current user authorized?
# 4) Is this action reversible / sensitive?
# 5) Did execution succeed?
# 6) What observation is safe to return to the model?
```

Congratulations

You now have the core of a native ReAct-style agent: Gemini can choose actions, but your Python application remains responsible for schemas, validation, execution, observations, limits, and safety.